

☒ Webアプリケーション開発 ITリテラシー理解度チェック

タスク管理アプリ「TaskManagement」の開発を通じて身につけるべき

第1章 インターネットとネットワークの仕組み（12問）

Q1. インターネットとは

「インターネット」と「Webブラウザで見るホームページ」は同じものですか？違うとすれば、どう違いますか？

「インターネット」と「Webブラウザで見るホームページ」は違うものです。ホームページはインターネット上で動くサービスの1つです。Webブラウザ（Google ChromeやSafariなど）がインターネット上のサーバーに置かれているデータを取得・解釈して、画面に表示したものがホームページです。この流れは一部例外はあるものの、インターネットなしでは起きません。

Q2. IPアドレスとポート番号

このプロジェクトではフロントエンドを localhost:5173、バックエンドを localhost:8080 で起動しています。

・localhostとは何を指しますか？

・:5173 や :8080 のようなポート番号はなぜ必要ですか？

localhostとは、自分自身のコンピュータを指す特別なホスト名です。ブラウザで localhost にアクセスする、ローカル環境と呼ばれる状況では、インターネットなしでWebページを表示できます。また、外部から自分の開発中のコードにアクセスされないというメリットがあることから、主に開発中のテストに使います。

:5173 や :8080 のようなポート番号が必要なのは、同じ番号は1つのサービスしか使えないためです。ポート番号とは、1台のPCで複数のサービスを同時に動かすための「窓口番号」のことを指します。ポート番号が同じとき、先に起動した方がその番号を占有し、後から同じポートで起動した方はエラーになります。よって、フロントとバックはそれぞれ別の番号を使います。

Q3. DNS

ブラウザのアドレスバーに `https://www.example.com`

と入力してからページが表示されるまでに、DNSはどのような役割を果たしますか？

DNSはドメイン名をIPアドレスに変換する役割を持っています。ユーザーはドメイン名を使ってインターネットを利用しますが、コンピューターがデータを送受信する際には、IPアドレスが必要です。DNSがなければ、ユーザーがドメイン名をアドレスバーに入力しても、コンピューターはそれを認識できず、目的のサーバーに接続することができません。その場合、ユーザーが直接IPアドレスを入力する必要がありますが、数字の羅列は覚えにくく、非常に不便です。DNSはこの変換を自動で行うことで、ユーザーが意識せずにインターネットを利用できるようにしています。

Q4. HTTPとは

HTTPとは何の略で、どのような役割を持つものですか？「プロトコル」という言葉の意味も含めて説明してください。

HTTPはHyperText Transfer Protocolの略です。プロトコルとは通信のルール・手順の取り決めのことで、HTTPはその役割として、ブラウザがサーバーにリクエストを送り、サーバーがHTMLやデータをレスポンスとして返すやり取りの形式を定めています。コンピュータ同士の通信では、「どんな形式でデータを送るか」「エラーのときはどうするか」といったルールを事前に決めておく必要があります。

Q5. HTTPとHTTPSの違い

HTTPとHTTPSの違いは何ですか？なぜ現在のWebサイトはほぼすべてHTTPSを使用していますか？

HTTPS は HTTP に S (Secure) が加わったもので、通信内容を暗号化してセキュリティを高めたバージョンです。現在のWebサイトのほとんどはHTTPSを使用しています。その理由の1つとして、HTTPはデータが平文でそのまま送受信され、途中で第三者に盗み見られる可能性があります。パスワードやクレジットカード番号などの個人情報が盗聴・改ざんされるリスクがあるため、セキュリティの高いHTTPSが標準となっています。

Q6. リクエストとレスポンス

Webにおける「リクエスト」と「レスポンス」の関係を、日常生活の例えを使って説明してください。

今日の晩御飯は何かを親に聞くパターンで説明してみようと思います。

「今日の晩御飯はなに？」（リクエスト）、親が冷蔵庫や棚を見て献立を考える（サーバーの処理）、「今日はカレー」（レスポンス）

Q7. HTTPメソッド

このアプリのAPIでは GET / POST / PUT / DELETE / PATCH を使っています。これらの「HTTPメソッド」とは何ですか？日常生活の例えで、それぞれの違いを説明してください。

HTTPメソッドは「サーバーに対して何をしたいかを伝える言葉」で、5つは以下の操作に対応しています。

GET - データを読む POST - 新しく作る PUT - 全部置き換える DELETE - 消す PATCH - 一部だけ置き換える

日常生活「メソッドはスマホの連絡先の操作」で例えてみようと思います。

GET - 連絡先を検索・表示する POST - 新しい連絡先を追加登録する PUT - 連絡先を丸ごと別の内容で上書きする DELETE - 連絡先を削除する PATCH - 電話番号だけ変えるなど、一部だけ修正する

Q8. HTTPステータスコード

サーバーからの返事には「ステータスコード」という3桁の数字が含まれます。以下のコードはそれぞれ何を意味しますか？（200 / 201 / 400 / 404 / 500）

200 - OK：リクエスト成功。サーバーが正常に処理し、結果を返した状態です。最も一般的な成功レスポンスです。

201 -

Created：リソースの作成に成功。主にPOSTリクエストで新しいデータが作成されたときに返されます。

400 - Bad Request：クライアント側のリクエストに問題がある状態。送ったデータの形式が間違っていたり、必須項目が欠けていたりする場合です。

404 - Not Found : 指定したリソースが存在しない状態。URLが間違っていたり、すでに削除されたページにアクセスしたときに返されます。

500 - Internal Server Error : サーバー側で予期しないエラーが発生した状態。クライアントには責任がなく、サーバーのプログラムやインフラに問題がある場合です。

Q9. JSONとは

フロントエンドとバックエンド間でデータをやり取りする際に「JSON」という形式が使われています。JSONとは何ですか？なぜ広く使われていますか？

JSON (JavaScript Object Notation) は、データを表現するためのテキストベースのフォーマットです。JSONはもともとJavaScriptの構文から生まれたため、Webブラウザとの相性がよく、テキスト形式なので軽量で転送効率が良いです。また、シンプルで読みやすく、JavaScript・Python・Java・Ruby・PHPなど、あらゆる言語にJSONのパースャーが標準で備わっています。このことから、フロントエンドとバックエンドの共通言語として定着しました。

Q10. APIとは

このアプリではフロントエンドがバックエンドのAPIを呼び出してデータを取得しています。APIとは何ですか？なぜ直接データベースにアクセスしないのですか？

API (Application Programming Interface) は、決められた形式でデータや機能をやり取りするための仕組みのことです。フロントエンドが決まった形式でリクエストを送り、APIが決まった形式でレスポンスを返します。直接データベースにアクセスしない最も理由は「セキュリティ」です。フロントエンドのコードはブラウザで動くため、ユーザーが中身を見ることができます。もし直接DB接続していたら、接続情報（パスワードなど）が丸見えになり、誰でもデータを盗んだり壊したりできてしまいます。

Q11. CORSとは

フロントエンド (5173) からバックエンド (8080) にアクセスする際、「CORS」という仕組みが関係します。CORSとは何のためにありますか？

CORS (Cross-Origin Resource Sharing) は、ブラウザが「異なるオリジン」へのリクエストを制限する仕組みです。オリジンとは、プロトコル + ホスト + ポート の組み合わせたものです。

ブラウザは、あるページが読み込まれたオリジン (例 : localhost:5173) と、リクエスト先のオリジン (例 : localhost:8080) を比較します。この2つが一致しない場合、ブラウザは「クロスオリジンリクエスト」と判断し、デフォルトでブロックします。ブロックする一番の理由は、ログイン済みのユーザーを悪意あるサイトから守るためです。CORSはバックエンドに「このサイトからのリクエストを許可するか」を判断させることで、意図しないオリジンからの操作を防ぐ仕組みになっています。

Q12. クライアントとサーバー

「クライアント」と「サーバー」とは何ですか？このアプリの構成の中で、それぞれどれがクライアントでどれがサーバーですか？ (ブラウザ / バックエンド / データベース)

クライアントは「サービスを要求する側」、サーバーは「サービスを提供する側」です。この関係は相対的なもので、固定ではありません。

アプリの構成は「ブラウザ → バックエンド → データベース」となっていて、ブラウザ ↔ バックエンドの関係の時、クライアントはブラウザ、サーバーはバックエンド、バックエンド ↔ データベースの関係の時、クライアントはバックエンド、サーバーはデータベースとなります。

第2章 Webアプリケーションの構造（8問）

Q13. フロントエンドとバックエンド

「フロントエンド」と「バックエンド」の違いを説明してください。このアプリではそれぞれ何を担当していますか？

フロントエンドはユーザーが直接見て触れる部分、バックエンドはその裏側で動く仕組みのことです。

違いとしては、フロントエンドはウェブサイトやアプリの画面に表示されるすべてが対象で、バックエンドはサーバー側で動くデータ処理や業務ロジックが対象という点です。

このアプリでは、フロントエンドはボード全体の状態管理・画面の描画、カラムのUI、カード1枚のUI、検索UIの描画、バックエンドへのHTTPリクエストを全て集約（fetchを呼ぶ）を担当しています。

バックエンドはHTTPリクエストを受け取り適切なServiceに渡す窓口、ビジネスロジック、PostgreSQLへの読み書き（Spring Data JPA）、DBテーブルに対応するオブジェクト（CardEntity, ColumnEntity）、フロントエンドとやり取りするデータの形を担当しています。

Q14. 3層構造

このアプリはフロントエンド・バックエンド・データベースの3層構造です。なぜこのように役割を分けるのですか？1つのプログラムですべてをやらない理由を説明してください。

役割を分ける理由は、責任の分離のためです。フロントエンド、バックエンド、DBは異なる専門性を持っています。それらが独立することによって、セキュリティを守りやすくなります。重要なDBの中身をインターネットに晒さないよう、バックエンドが最終門番となり「このユーザーはこのデータを見ていいのか？」を一箇所でチェックすることが出来ます。

また、アクセスが増えたとき、バックエンドのサーバーだけを増やして対応できます。すべてが1つのプログラムだと、全部まとめて増やすしかなく非効率になります。層ごとに独立しているので、影響範囲が限定されます。

Q15. データの流れ

ユーザーが「新しいタスクを作成」ボタンを押してから、データベースに保存される・画面に反映されるまでの流れを順を追って説明してください。

「新しいタスクを作成」ボタンを押し、カンバンに入力した内容（タスクのタイトルなど）に問題がないかをまずチェックします。チェック後、インターネット経由でデータがサーバーに送信されます。返事が来るまでの間、ローディング表示が出ます。サーバーが受け取ると、「このユーザーは本当に正しいユーザーか？」などを確認があります。確認されたデータはデータベースに保存され、ここにタスクが書き込まれます。保存ができたという返事がサーバーから戻ってきたら、画面のリストに新しいタスクが追加・表示されます。

Q16. エンドポイントとURL設計

このアプリには `/api/tasks` や `/api/tasks/{id}`

のようなAPIがあります。このようなURL設計の考え方を説明してください。{id} の部分は何ですか？

このURL設計はRESTという考え方に基づいています。URLで「何のデータか」を表し、GETやDELETEなどのHTTPメソッドで「何をするか」を表します。

{id}はパスパラメータと呼ばれ、特定の1件を識別するための値が入ります。例えば /api/tasks/1 ならIDが1のタスク、/api/tasks/42 ならID42のタスクを指します。/api/tasks だけでは「どのタスクか」が特定できないため、1件を取得・更新・削除する際にこの{id}が必要になります。

Q17. 同期処理と非同期処理

「同期処理」と「非同期処理」の違いを、日常生活の例えで説明してください。なぜWeb通信では非同期処理が必要ですか？

同期処理 = 電話、非同期処理 = メール / LINEとして説明します。

電話は電話をかけると、相手が出るまでずっと待ちます。その間、他のことは何もできません。一方、メールやLINEはメッセージを送ったら、すぐに他のことができます。

非同期処理が必要なのは、Web通信には時間がかかるからです。仮に同期処理でWeb通信をすると、通信中はJavaScriptがフリーズし、ボタンが押せなかったり、スクロールができなかったりします。UIの再描画も止まり、画面が固まったように見え、ユーザーが操作できず「壊れた？」と感じてページを閉じるなどの問題が生じるため、待っている間も画面を生かし続けるために非同期処理が不可欠です。

Q18. SPA（シングルページアプリケーション）

このアプリはReactで作られたSPAです。従来のWebサイト（ページ遷移するもの）と何が違いますか？

SPAと従来のWebサイト（MPA）の最大の違いは、ページ遷移の仕組みにあります。

MPAではリンクをクリックするたびにサーバーへリクエストを送り、HTMLを丸ごと受け取ってページ全体を再描画します。一方SPAは、最初に一度だけHTMLとJavaScriptを読み込み、以降はJSが画面の必要な部分だけを書き換えます。URLは変わっても、ページ全体のリロードは発生しません。

これによりSPAはページ遷移が瞬時に行われ、アプリのようなスムーズな操作感を実現できます。サーバーとの通信もHTMLではなくJSONのAPIだけになるため、サーバー負荷も下がります。ただし、初回にJavaScriptのバンドルファイルをまとめて読み込む分、最初の表示はMPAより遅くなることがあります。

Q19. 入力チェック（バリデーション）の多重防御

入力チェックはフロントエンドとバックエンドの両方で行うべきと言われています。なぜ片方だけでは不十分なのですか？

片方だけでは不十分な理由は、それぞれが担う役割がまったく異なるためです。

フロントエンドのバリデーションはブラウザ上、つまりユーザーの手元で動作します。そのためコードの読み取りや改ざんが可能で、開発者ツールでJavaScriptを無効化したり、curlやPostmanでHTMLを介さず直接APIを叩いたりすることで容易に回避できます。フロントエンドのチェックは「UIの見た目上の制約」にすぎず、セキュリティ上の保証にはなりません。

一方、バックエンドだけにバリデーションを任せると、ユーザー体験が悪化します。入力ミスサーバーに送信して初めてエラーが返ってくるため通信遅延が生じ、メールアドレスの形式ミスのような単純なチェックにもサーバーリソースを消費してしまいます。

つまりフロントエンドは「ユーザーへの即時フィードバック」、バックエンドは「セキュリティとデータ整合性の保証」という異なる責務を持ちます。バックエンドは常に「フロントのチェックが存在しない前提」で実装することが鉄則であり、両方を組み合わせることで快適さと安全性が初めて両立します。

Q20. エラーハンドリング

API呼び出しが失敗した場合（ネットワーク障害やサーバーエラーなど）、アプリはどのように対処すべきですか？

API呼び出しが失敗した場合、エラーの種類によって対処方法は大きく3つに分かれます。

- ① ネットワーク障害・5xxサーバーエラー：接続タイムアウトやサーバー側の一時的な障害は、時間をおいて再試行することで解決できる場合がほとんどです。指数バックオフという戦略を使うのが定石です（1回目は1秒待ち、2回目は2秒、3回目は4秒と倍にしていく方法）。
- ② 4xxクライアントエラー：400番台のエラーは、リクエスト自体に問題があるためサーバーが拒否しているものです。同じリクエストをリトライしても結果は変わらないので、原因を特定して対応を変える必要があります。
- ③ サーキットブレーカー：エラーが一定回数以上続いた場合、しばらくの間APIへのアクセス自体を停止するパターンです。どのエラーでも共通して重要なのは、ユーザーに適切なフィードバックを返すことです。

第3章 データベースの基礎（8問）

Q21. データベースとは

データベースとは何ですか？ExcelやCSVで保存する方法と比べて何が優れていますか？

データベースとは、データを整理・管理・検索しやすい形で保存する仕組みです。単なるファイルではなく、専用のソフトウェア（DBMS）が管理するため、大量のデータを効率よく扱えます。代表例として、MySQL、PostgreSQLなどがあります。

ExcelやCSVで保存する方法は、個人の小規模なデータ管理には向いていますが、大量データの処理速度、複数人での同時アクセス、データの正確性、途中で失敗しても処理を取り消せる（トランザクション）、セキュリティなどの点において、データベースで保存する方が優れています。

Q22. テーブル・行・列

以下をExcelのシートに例えて説明してください。テーブル / 行（レコード） / 列（カラム）。tasksテーブルで具体例も挙げてください。

- ・ テーブル：「tasks」という名前のシートそのもの＝Excelのシート1枚
- ・ 行（レコード）：実際に登録された1件分のデータ＝Excelの1行分のデータ
- ・ 列（カラム）：データの種類・項目名を定義する＝Excelの列ヘッダー

例：id=1 / title=ログイン機能を実装する / status=TODO / column_id=1

Q23. 主キー（PRIMARY KEY）

tasksテーブルの id は主キーです。主キーとは何ですか？なぜ必要ですか？

主キーとは、「各行を一意に識別するための列」です。各行に振られた絶対に重複しない管理番号のようなもので、この番号を指定するだけで、どの行か必ず1件に絞り込めます。

主キーが必要な理由の1つは、特定の1件を正確に操作するためです。タイトルで検索すると「同名タスク」が複数ある場合に困りますが、id=2なら必ず1件に絞れます。もう1つは、テーブル間の関係を作るためです。column_idのように、他のテーブルの主キーを参照することで「このタスクはどのカラムに属するか」を表現できます。

Q24. CRUD操作

CRUDとは何の略ですか？それぞれがこのアプリのどの機能に対応するか説明してください。

CRUDとは、Create/Read/Update/Deleteの略です。

Create（作成）：新しいデータをテーブルに追加します。

Read（読み取る）：テーブルからデータを取得して画面に表示します。

Update（更新）：既存の行のデータを変更します。

Delete（削除）：テーブルから行を削除します。

Q25. データの制約

tasksテーブルでは NOT NULL と DEFAULT 'medium' が設定されています。これらは何のためにありますか？

どちらも「おかしなデータがDBに入るのを防ぐ」ための制約です。アプリ側のバリデーションと二重で守ることでデータの整合性を保ちます。

- ・NOT NULL：「この列を空欄にしてはいけない」というルール。タイトルのないタスクが存在すると、画面表示やデータ検索が壊れます。

- ・DEFAULT 'medium'：「値が指定されなかったときに自動でセットする初期値」の設定。カード追加のたびに優先度を毎回入力するのは手間なので、合理的なデフォルトを持たせることでコードをシンプルに保てます。

Q26. インデックス

インデックスとは何ですか？本の索引に例えて説明してください。

インデックスとは、データを素早く見つけるための「対応表」です。本の索引で例えると、「アルゴリズム → 42ページ」のように、キーワードとページ番号が対応しています。おかげで本を最初から読まなくても、目的の情報にすぐたどり着けます。

データベースのインデックスもまったく同じ仕組みです。100万件の顧客データから「田中さん」を探するとき、インデックスがなければ先頭から1件ずつ確認しなければなりませんが、インデックスがあれば一瞬で見つかります。

ただし、インデックスはその分だけストレージを余分に使います。データを追加・更新するたびに索引も書き直す必要があるため、書き込みが少し遅くなるというトレードオフもあります。

Q27. トランザクション

トランザクションとは何ですか？タスクを別カラムへドラッグ移動する操作で、なぜ重要ですか？

トランザクションとは、一連の操作をひとまとまりとして扱う仕組みです。その特性はACIDという4つの性質で表されます。原子性（Atomicity）：全部成功するか全部失敗するかのどちらか。一貫性（Consistency）：操作の前後でデータの整合性が保たれる。分離性（Isolation）：複数の処理が同時に走っても互いに干渉しない。永続性（Durability）：操作が成功した後に障害が起きてもデータが消えない。

タスクをカラムへドラッグ移動する操作では複数の更新が連続して発生します。トランザクションがない場合、途中で障害が起きると、タスクがどのカラムにも属さない状態のままデータが残ってしまいます。トランザクションを使うことで、これらの操作がすべて成功するか、あるいはすべてロールバックされるかのどちらかに保たれます。

Q28. SQLとORM

SQLとはどのようなものですか？ORMの役割とは何ですか？

SQLとは「Structured Query Language」の略で、リレーショナルデータベースを操作するための標準的な言語です。データの取得・追加・更新・削除といったCRUD操作や、テーブルの作成・管理を行うために使われます。

ORM (Object-Relational Mapper) とは、SQLを直接記述せずにデータベースを操作できるようにするライブラリです。ORMの主な役割は3つあります。①抽象化：DBの構造をオブジェクトとして隠蔽し、コードの可読性を高めます。②安全性の向上：SQLインジェクションへの対策が自動的に施されます。③移植性：使用するデータベースの種類を変更する際もコードの修正を最小限に抑えられます。

第4章 開発プロセスとチーム開発（4問）

Q29. バージョン管理 (Git)

Gitなどのバージョン管理システムとは何ですか？なぜ必要ですか？使わないとどんな問題が起きますか？

バージョン管理システムとは、コードやファイルの変更履歴を記録・管理するためのシステムです。Gitは其中でも現在最も広く使われているツールです。いつ、誰が、何をどのような理由で変更したかを正確に記録し、必要に応じて過去の任意の状態に戻ることができます。

必要な理由：①変更履歴の追跡、②過去の状態への復元、③チームでの並行作業の実現、④安全な実験環境の提供。

使わない場合の問題：誰かが別のメンバーの変更を誤って上書きしてしまう、バグが発生しても原因追跡が困難、「final.py」「final2.py」「final_本当の最終.py」といったファイルが乱立する、パソコンの故障で全データを失うリスクなど。

Q30. ブランチとPull Request

ブランチとは何ですか？Pull Request の目的は何ですか？なぜ全員が main に直接変更しないのですか？

ブランチとは、コードベースの独立したコピーのようなものです。ブランチを使うことで、メインのコードに影響を与えることなく、新機能の開発や実験を安全に行うことができます。

Pull Request (PR) は、「この変更を main にマージしてください」というリクエストです。単なるマージ依頼にとどまらず、チームメンバーがコードをレビューしてコメントできる場であり、CI/CDを走らせるタイミングにもなります。

全員がmainに直接変更してしまうと、バグがそのまま本番環境に出てしまったり、他の人の作業と衝突しやすくなったりします。ブランチとPRを使うことで、レビューを通じて問題を事前に発見でき、各自が独立して作業できます。

Q31. コミットメッセージの重要性

例：feat: ログイン機能を追加。良いコミットメッセージを書くことがなぜ重要ですか？

コミットメッセージはコードと同じく、チームへのコミュニケーションです。コードを読めば「何を」変えたかはわかりますが、「なぜ」変えたかはコミットメッセージにしか残りません。

良いメッセージが積み重なると、git logがそのままプロジェクトの設計書になります。変更の流れを時系列で追えるため、新しいメンバーのオンボーディングや仕様確認の際にも役立ちます。

バグ調査の場面でもgit blameやgit bisectで問題箇所を特定できても、メッセージが貧乏ければ「なぜその変更をしたのか」を調べ直す手間が生じます。コミットメッセージは未来の自分を含むチーム全員への手紙だと考えると、自然と丁寧に書けるようになります。

Q32. なぜ技術的な仕組みを理解すべきか

AI時代においてコードはAIが書けるようになりました。それでもネットワーク・データベース・Webアプリケーションの仕組みを理解する必要がある理由を、あなたの考えで述べてください。

AIはとても優秀で、ITやプログラム初心者の私のわからないことや疑問に答えてくれる「ツール」です。あくまで聞いたことに答えてくれるだけで、質問の質によってAIの回答は異なってくるものだと思います。その回答を鵜呑みにして信じ込み自分の知識として取り入れてしまい、それが間違っていたとしてもITの知識がないので気づくことはないでしょう。

ここで大事なのは、ネットワーク、データベース、Webアプリケーションの仕組みに対する理解と知識です。これらに対する理解・知識があるだけで質問の質が変わります。AIが普及したことで、難しく時間のかかるコードを自動で書いてくれるようになりました。実装をAIが担ってくれるからこそ、人間に残された役割は設計・判断・検証であり、そこに仕組みの理解が直結します。

だからこそ人間に求められるのは、正しく問いを立て、AIの出力を判断する力です。ネットワーク・データベース・Webアプリケーションの仕組みへの理解は、その土台になると考えています。